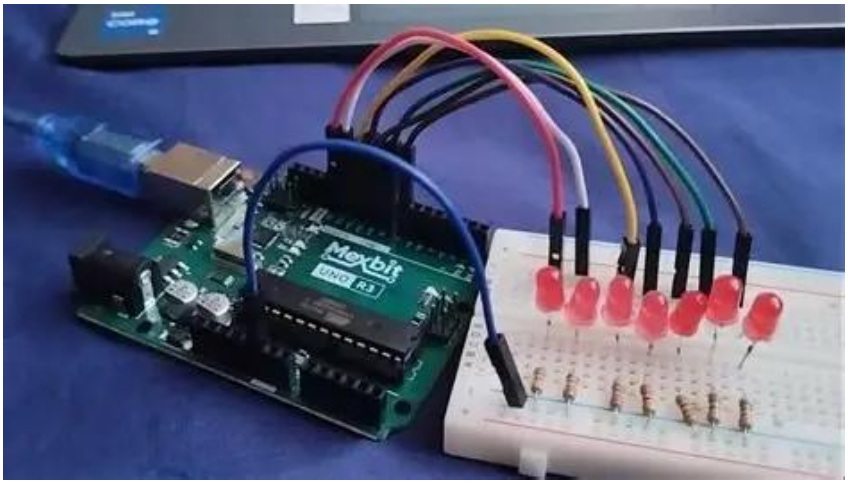
UNIVERSIDAD AUTONOMA DE TAMAULIPAS.

UNIDAD ACADEMICA MULTIDISCIPLINARIA DEL MANTE.

ARQUITECTURA DE EQUIPOS DIGITALES.

LABORATORIO 05 – 08



Nombre: Hernández Castro Abigail.

Docente: López Piña Daniel.

23 de abril del 2026.

LABORATORIO 05. Comunicación Serial UART: Interfaz entre MCU y PC

Objetivo:

Estableceremos una comunicación bidireccional UART entre un microcontrolador y una PC, enviando datos estructurados y procesando comandos recibidos desde el monitor serial.

Materiales:

• Arduino UNO/Nano o ESP32 DevKit

• Cable USB

• Arduino IDE 2.x instalado

• Monitor serial integrado en Arduino IDE

• Opcional: PuTTY o Termite para comunicación serial avanzada

Procedimiento:

1. Conectar el Arduino al PC. Abrir Arduino IDE, seleccionar la placa y puerto correctos.

2. Crear un nuevo sketch. Copiar el código de ejemplo de este laboratorio.

3. Compilar y cargar el sketch al Arduino.

4. Abrir el Monitor Serial (Ctrl+Shift+M o Herramientas > Monitor Serial). Configurar la velocidad a 115200 bps y seleccionar 'Ambos NL & CR'.

5. Observar los mensajes de inicio que el Arduino envía automáticamente al arrancar.

6. En el campo de texto del monitor serial, escribir 'LED_ON' y presionar Enter. Verificar que el LED integrado enciende.

7. Escribir 'LED_OFF' y verificar que el LED se apaga.

8. Escribir 'STATUS' y verificar que el Arduino responde con su estado actual.

9. Escribir 'BLINK:3' y verificar que el LED parpadea 3 veces rápido como

confirmación.

10. Intentar enviar un comando invalido (ej. 'HOLA') y verificar el mensaje de error.

Código:

```
// lab05_uart_comandos.ino
// Comunicacion UART bidireccional con parser de comandos

const int LED_PIN = 13;
bool ledEstado = false;
String bufferCmd = "";

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  // Mensaje de bienvenida con datos del sistema
  Serial.println("==============================");
  Serial.println("Sistema UART - Arduino/ATmega");
  Serial.println("Baud rate: 115200 | Formato: 8N1");
  Serial.println("Comandos: LED_ON, LED_OFF, STATUS, BLINK:N");
  Serial.println("==============================");
}

void loop() {
  // Leer datos disponibles en el buffer UART
  while (Serial.available() > 0) {
    char c = Serial.read();
```

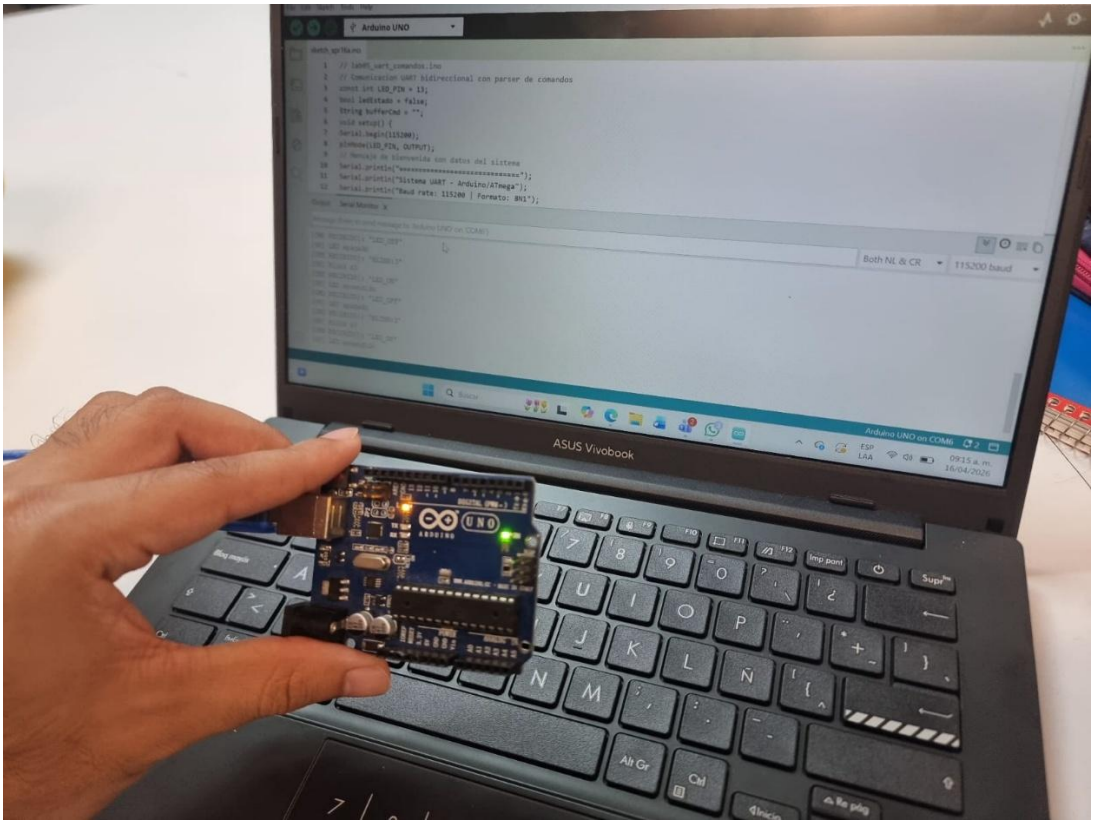
```
    if (c == '\n') {
      procesarComando(bufferCmd);
      bufferCmd = "";  // limpiar buffer
    } else if (c != '\r') {
      bufferCmd += c;  // acumular caracteres
    }
  }
}

void procesarComando(String cmd) {
  cmd.trim();  // eliminar espacios al inicio/fin
  Serial.print("[CMD RECIBIDO]: '"); Serial.print(cmd); Serial.println("'");

  if (cmd == "LED_ON") {
    ledEstado = true;
    digitalWrite(LED_PIN, HIGH);
```



```
    Serial.println("[OK] LED encendido");
  }
  else if (cmd == "LED_OFF") {
    ledEstado = false;
    digitalWrite(LED_PIN, LOW);
    Serial.println("[OK] LED apagado");
  }
  else if (cmd == "STATUS") {
```

```
    Serial.println("--- STATUS ---");
    Serial.print("LED: "); Serial.println(ledEstado ? "ON" : "OFF");
    Serial.print("Uptime: "); Serial.print(millis()/1000);
Serial.println(" s");
    Serial.print("Free SRAM aprox: "); Serial.print(freeRam());
Serial.println(" bytes");
  }
  else if (cmd.startsWith("BLINK:")) {
    int veces = cmd.substring(6).toInt();
    for (int i = 0; i < veces; i++) {
      digitalWrite(LED_PIN, HIGH); delay(150);
      digitalWrite(LED_PIN, LOW);  delay(150);
    }
    Serial.print("[OK] Blink x"); Serial.println(veces);
  }
  else {
    Serial.print("[ERROR] Comando desconocido: '"); Serial.print(cmd);
Serial.println("'");
  }
}

int freeRam() {
  extern int __heap_start, *__brkval;
  int v;
  return (int) &v - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);
```

Resultado:



Conclusión:

Se logró establecer una comunicación entre el microcontrolador y la computadora, permitiendo enviar y recibir información de manera sencilla. Gracias a esto, es posible controlar el sistema y responder a los comandos desde el monitor serial, haciendo más fácil la interacción y el manejo de los datos.

LABORATORIO 07: PWM y Control Analógico: Dimmer de LED y Control de Servo

Objetivo:

Generaremos señales PWM con un microcontrolador para controlar el brillo de un LED y la posición de un servomotor, comprendiendo la relación entre el ciclo de trabajo y el efecto físico resultante.

Materiales:

• Arduino UNO, Nano o ESP32 (cualquier plataforma con PWM)

• LED de cualquier color + resistencia 220 ohm

• Servomotor SG90 o MG90S (servo de 9g, muy económico)

• Potenciómetro de 10 kohm (para control manual del servo)

• Protoboard y jumpers

• Multímetro (opcional, para medir el voltaje promedio del PWM)

Procedimiento:

1. Armar el circuito: LED con resistencia 220 ohm en pin 9 (PWM). Servomotor: VCC (rojo) a 5V, GND (negro/café) a GND, señal (naranja/amarillo) a pin 10. Potenciómetro: extremos a 5V y GND, wiper (cursor) a pin A0.

2. Cargar el sketch del laboratorio. Abrir el Monitor Serial a 115200 bps.

3. Observar la rampa de brillo del LED: debe ir de apagado a plena intensidad y volver suavemente (efecto 'respiración').

4. Al mismo tiempo, el servo debería moverse de 0 a 180 grados y regresar sincronizado con el LED.

5. Girar el potenciómetro lentamente. El servo debería seguir la posición del potenciómetro. El LED debería mostrar el brillo proporcional.

6. Usar el Monitor Serial para enviar comandos: 'D:128' para poner el LED al 50% de

brillo, 'S:90' para mover el servo a 90 grados, 'AUTO' para iniciar la demo automática.

7. Con el multímetro en modo VDC, medir el voltaje en el pin del LED mientras cambia el duty cycle. Verificar que el voltaje promedio cambia proporcionalmente.

8. Probar con el potenciómetro al máximo, al centro y al mínimo. Medir el voltaje en A0 para cada posición y comparar con la lectura del ADC reportada por el serial.

Código:

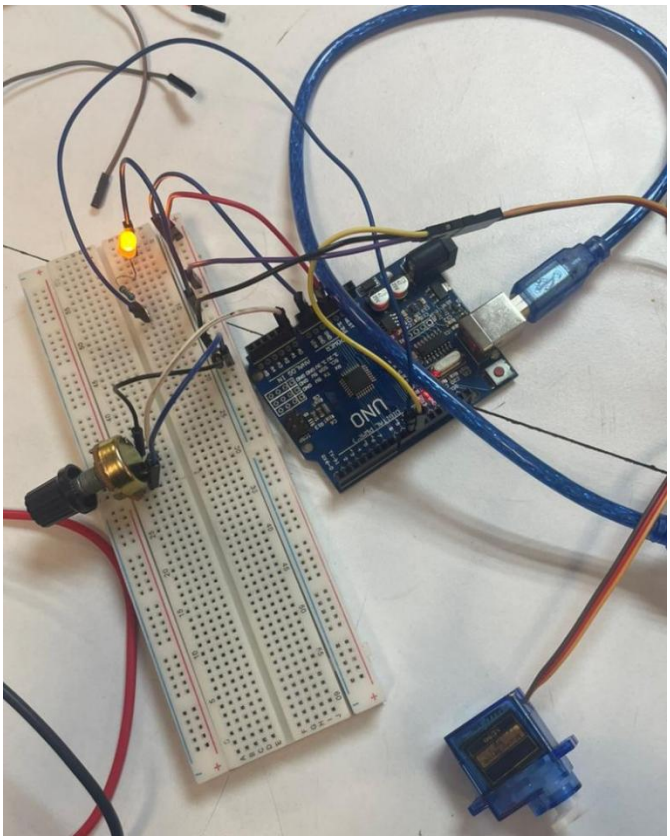
```
// lab07_pwm_servo_dimmer.ino
#include <Servo.h>

const int PIN_LED   = 9;    // pin PWM para LED
const int PIN_SERVO = 10;   // pin PWM para servo
const int PIN_POT   = A0;   // entrada analogica del potenciometro

Servo miServo;
bool modoAuto = true;
int brightnessTarget = 0;
int servoTarget = 90;
String bufferCmd = "";

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  miServo.attach(PIN_SERVO);
  miServo.write(90);  // posicion inicial
```

```
  Serial.println("Lab 07 - PWM + Servo + ADC");
  Serial.println("Comandos: D:0-255 | S:0-180 | AUTO | POT");
}

void loop() {
  // Leer comandos seriales
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n') { procesarComando(bufferCmd); bufferCmd = ""; }

    else if (c != '\r') bufferCmd += c;
  }

  if (modoAuto) {
    // Demo: rampa de brillo y movimiento de servo
    static int val = 0;
    static int dir = 1;
    static unsigned long ultimo = 0;
    if (millis() - ultimo > 10) {
      ultimo = millis();
      val += dir * 2;
      if (val >= 255) { val = 255; dir = -1; }
      if (val <= 0)   { val = 0;   dir =  1; }
      analogWrite(PIN_LED, val);
      int grados = map(val, 0, 255, 0, 180);
      miServo.write(grados);
```

```
    }
  } else {
    // Modo manual: potenciometro controla servo y LED
    int lecturaPot = analogRead(PIN_POT);  // 0-1023
    int grados = map(lecturaPot, 0, 1023, 0, 180);
    int brillo  = map(lecturaPot, 0, 1023, 0, 255);
    miServo.write(grados);
    analogWrite(PIN_LED, brillo);
    static unsigned long ultimoReport = 0;
    if (millis() - ultimoReport > 500) {
      ultimoReport = millis();
      float voltaje = lecturaPot * (5.0 / 1023.0);
      Serial.print("ADC="); Serial.print(lecturaPot);
      Serial.print(" V="); Serial.print(voltaje, 2);
      Serial.print(" Servo="); Serial.print(grados);
      Serial.print("deg  LED="); Serial.println(brillo);
    }
  }
}

void procesarComando(String cmd) {
  cmd.trim();
  if (cmd == "AUTO")  { modoAuto = true;  Serial.println("[OK] Modo
automatico"); }
  else if (cmd == "POT") { modoAuto = false; Serial.println("[OK] Modo
potenciometro"); }
```

```
  else if (cmd.startsWith("D:")) {
    int v = cmd.substring(2).toInt();
    v = constrain(v, 0, 255);
    analogWrite(PIN_LED, v);
    modoAuto = false;
    Serial.print("[OK] LED duty="); Serial.println(v);
  }
  else if (cmd.startsWith("S:")) {
    int g = cmd.substring(2).toInt();
    g = constrain(g, 0, 180);
    miServo.write(g);
    modoAuto = false;
    Serial.print("[OK] Servo="); Serial.print(g); Serial.println(" grados");
  }
}
```

Resultado:



Conclusión:

Se pudo comprobar que mediante señales PWM es posible controlar tanto el brillo de un LED como el movimiento de un servomotor. Al ajustar el ciclo de trabajo, se logran cambios visibles y precisos, lo que demuestra cómo una señal digital puede influir directamente en el comportamiento de componentes físicos.